1A

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:126:45: warning: '.tmp' directive output may be truncated writing 4 bytes into a region of size between 1 and 512 [-Wformat-truncati
on=]
  126 |     snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", path);
      |                                             ^~~~
object.c:126:5: note: 'snprintf' output between 5 and 516 bytes into a destination of size 512
  126 |     snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", path);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$
```

1B

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$
```

2A

```
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ xxd .pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2068  tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 0066 2246 63d2 3e6f  ello.txt.f"Fc.>o
00000020: 4d9d e9e2 c7e6 d876 4305 a92a 3830 a1a5  M......vC..*80..
00000030: 2d3d 5f4a a800 7b5c 39                   -=_J..{\9
```

3A

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ nano pes.c
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ make pes
rm -rf .pes
./pes init
echo "hello world" > file1.txt
echo "goodbye world" > file2.txt
./pes add file1.txt file2.txt
./pes status
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
    staged: file1.txt
    staged: file2.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    untracked: test_objects.c
    untracked: pes.c
    untracked: .gitignore
    untracked: commit.h
    untracked: index.h
    untracked: Makefile
    untracked: test_objects
    untracked: tree.h
    untracked: tree.c
    untracked: pes.h
    untracked: test_sequence.sh
    untracked: index.c
    untracked: object.c
    untracked: commit.c
    untracked: test_tree.c
    untracked: README.md
    untracked: test_tree
```

3B

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 17766575
26 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 17766575
26 6 file2.txt
```

4A

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ rm -rf .pes
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
./pes log
Initialized empty PES repository in .pes/
Committed: fa613ed26c19... Initial commit
Committed: 8e35895174b3... Add world
Committed: 410ec4125d09... Add farewell
commit 410ec4125d0930d125ee2af3766700d955a138e570a0b6e4e8e80725fb2b9431
Author: Bhavyaa Garg <PES1UG24CS116>
Date:   1776658467

    Add farewell

commit 8e35895174b3b4eeddb33a3e3a15d92b2e9fe6bcb6679dacd8181904137e2b6d
Author: Bhavyaa Garg <PES1UG24CS116>
Date:   1776658467

    Add world

commit fa613ed26c196a40b341c5c5743f664c717712805365d9b2addb30895a1a028f
Author: Bhavyaa Garg <PES1UG24CS116>
Date:   1776658467

    Initial commit
```

4B

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/41/0ec4125d0930d125ee2af3766700d955a138e570a0b6e4e8e80725fb2b9431
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8e/35895174b3b4eeddb33a3e3a15d92b2e9fe6bcb6679dacd8181904137e2b6d
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/fa/613ed26c196a40b341c5c5743f664c717712805365d9b2addb30895a1a028f
.pes/refs/heads/main
```

4C

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
410ec4125d0930d125ee2af3766700d955a138e570a0b6e4e8e80725fb2b9431
ref: refs/heads/main
```

make test-integration

```
bhavyaa@debian:~/PES1UG24CS116-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
    staged: file.txt
    staged: hello.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    (nothing to show)


--- First Commit ---
Committed: d1791a275437... Initial commit

Log after first commit:
commit d1791a2754377dc450041246a5a7f73acbd2a67409d545d9a09146128411b8d6
Author: Bhavyaa Garg <PES1UG24CS116>
Date:   1776658869

    Initial commit
```

```
--- Second Commit ---
Committed: 5da627a64613... Update file.txt

--- Third Commit ---
Committed: f81fe1e17ff3... Add farewell

--- Full History ---
commit f81fe1e17ff30991d111a9797315e843bf46c7d51d762766e1af6a30c6d7be17
Author: Bhavyaa Garg <PES1UG24CS116>
Date:   1776658869

    Add farewell

commit 5da627a64613a3d6c6f44c2c7dfb7e52d9d82fd54f2b774b06958f1ad24ef54e
Author: Bhavyaa Garg <PES1UG24CS116>
Date:   1776658869

    Update file.txt

commit d1791a2754377dc450041246a5a7f73acbd2a67409d545d9a09146128411b8d6
Author: Bhavyaa Garg <PES1UG24CS116>
Date:   1776658869

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f81fe1e17ff30991d111a9797315e843bf46c7d51d762766e1af6a30c6d7be17
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/5d/a627a64613a3d6c6f44c2c7dfb7e52d9d82fd54f2b774b06958f1ad24ef54e
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d1/791a2754377dc450041246a5a7f73acbd2a67409d545d9a09146128411b8d6
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f8/1fe1e17ff30991d111a9797315e843bf46c7d51d762766e1af6a30c6d7be17

=== All integration tests completed ===
```

Q5.1: How would you implement `pes checkout <branch>`?

To implement `pes checkout <branch>`, three things must happen:

1. **Update HEAD** — write `ref: refs/heads/<branch>` into `.pes/HEAD`.
2. **Update the working directory** — read the target branch's commit, walk its tree object, and overwrite every tracked file in the working directory with the blob content from the object store.
3. **Update the index** — replace the current index entries with the entries corresponding to the target tree.

What makes this complex: the working directory contains files not tracked by PES (untracked files) that must not be touched. Files present in the current branch but absent in the target must be deleted. Files modified but not committed create conflicts — checkout must detect these by comparing the working file's hash against the index entry and refuse if they differ.

Q5.2: How to detect a "dirty working directory" conflict?

For each file tracked in the index:
1. Compute the SHA-256 of the current working file content.
2. Compare it against the blob hash stored in the index entry.
3. If they differ, the file is locally modified and unstaged.

Then check if that same file path exists in the target branch's tree with a different blob hash. If both conditions are true — locally modified AND differs between branches — checkout must refuse with an error. This uses only the index (for the current staged state) and the object store (for both branch snapshots), with no other metadata needed.

Q5.3: What happens in detached HEAD state?

When HEAD contains a raw commit hash instead of `ref: refs/heads/<branch>`, new commits are written to the object store and HEAD is updated to point to the new commit hash directly. However, no branch pointer moves — the commits exist but are only reachable via HEAD itself.

If the user then checks out a branch, HEAD changes and the detached commits become unreachable — no branch or tag points to them. To recover, the user must know the commit hash (visible in the terminal history or via `pes log` before switching) and create a new branch pointing to it: create a file `.pes/refs/heads/<newbranch>` containing that hash, then update HEAD to reference it.

Q6.1: Algorithm to find and delete unreachable objects

Algorithm (mark-and-sweep):**

1. Start from all branch refs in `.pes/refs/heads/` and collect their commit hashes into a set called `reachable`.
2. For each commit hash in `reachable`, read the commit object, add its tree hash to `reachable`, add its parent hash (if any) to `reachable`, and enqueue the parent for processing.
3. For each tree hash in `reachable`, read the tree object, add all blob hashes and subtree hashes to `reachable`.
4. Walk all files in `.pes/objects/`. Any object whose hash is not in `reachable` is unreachable — delete it.

Data structure: A hash set (e.g., a C `uthash` table or a sorted array with binary search) for O(1) average lookup of reachable hashes.

Estimate for 100,000 commits, 50 branches: Assuming an average of 10 files per commit, each commit has 1 tree + ~10 blobs = ~11 objects. With deduplication across commits (unchanged files share blobs), a conservative estimate is visiting ~500,000–1,000,000 objects total across the mark phase.

Q6.2: Race condition between GC and concurrent commit

The race:
1. A commit operation calls `tree_from_index()`, which writes new blob objects to the object store. At this point, the blobs exist but no commit or branch points to them yet.
2. GC runs its mark phase — it traverses all branch refs and finds these new blobs unreachable (no commit references them yet).
3. GC deletes the blobs.
4. The commit operation calls `object_write()` for the commit object and `head_update()` — but the blobs it references are now gone. The repository is corrupted.

How Git avoids this: Git's GC uses a "grace period" — it only deletes objects older than 2 weeks by default (`gc.pruneExpire`). New objects are always recent, so they survive GC even if temporarily unreferenced. Git also writes a `gc.pid` lock file to prevent concurrent GC runs, and the commit operation completes in milliseconds, well within the grace window.